

SYSTEM DESIGN INTERVIEW

Design ChatGPT

LLM serving at scale: GPU pooling, KV-cache, streaming, safety, tiered routing

~ 45 minute read

© Study Guide — For personal use only

What's Inside

- 01 · Problem Statement
- 02 · Requirements
- 03 · Capacity Estimation
- 04 · High-Level Architecture
- 05 · GPU Pooling & Model Placement
- 06 · Continuous Batching
- 07 · KV-Cache Management
- 08 · Streaming Responses (SSE)
- 09 · Conversation Memory
- 10 · Tiered Routing
- 11 · Rate Limiting
- 12 · Safety Pipeline

- 13** · Cost Optimisations
- 14** · Inference Batching Diagram
- 15** · Failure Modes & Trade-offs
- 16** · Interview Playbook

01

Problem Statement

What we are building

Design a chatbot service powered by a large language model (**ChatGPT-style**) that answers user prompts in real-time, streams tokens back as they are generated, remembers the conversation, enforces safety, and serves hundreds of millions of users on a fleet of GPUs whose capex is measured in hundreds of millions of dollars.

Clarifying Questions

Question	Answer
Scale?	~200M monthly active users; ~1B prompts/day
Latency target?	<500ms time-to-first-token; ~50 tokens/sec stream
Models served?	Free (small), Paid flagship, Auto-route to thinking
Multi-turn?	Yes, conversations are stateful and durable
Streaming?	Yes, token-by-token via SSE / chunked HTTP
Safety?	Pre-filter input, post-filter output, separate classifier
Rate limits?	Token-bucket per user, org and tier
Tools/RAG?	Out of scope here; focus on raw chat serving

02

Requirements

Functional and non-functional

Functional Requirements

Requirement	Details
Send message	POST /v1/chat — accepts thread_id, prompt; returns SSE stream
List threads	GET /v1/threads — paginated user history
Resume thread	GET /v1/threads/:id — full message log
Stream tokens	Server-Sent Events; one event per token chunk
Cancel	DELETE /v1/chat/:request_id — abort decode mid-stream
Tier routing	Free → 8B model; Pro → 70B; Auto → thinking model
Safety	Refuse disallowed content; hide chain-of-thought

Non-Functional Requirements

Requirement	Target
TTFT (time-to-first-token)	p50 < 300ms, p99 < 1.5s
Throughput	12K queries/sec avg, ~50K/sec peak
Stream rate	~50 tok/sec/req for 70B model
Availability	99.9%; degrade to smaller model on overload
Durability	Conversation log retained 30 days hot, archive 1 year
GPU utilisation	> 70% sustained on flagship cluster
Cost ceiling	Free tier: ~\$0.005/query; Paid: ~\$0.05/query

03

Capacity Estimation

Math for serving an LLM at planet scale

Traffic

- MAU: **200M**; ~5 prompts/active-day → **~1B prompts/day**
- Average QPS: $1B / 86,400 \approx \text{~12K QPS}$
- Peak QPS (4× avg, US-EU overlap): **~50K QPS**
- Concurrent in-flight decodes (avg 10 sec/req): $12K \times 10 \approx \text{120K concurrent}$

Token Economics

- Avg context (system + history + prompt): **~200 tokens**
- Avg response: **~500 tokens**
- Total tokens generated/day: $1B \times 500 = \text{500B output tokens/day}$
- Total prefill tokens/day: $1B \times 200 = \text{200B input tokens/day}$

GPU Fleet

- Per-request decode rate (70B flagship): **~50 tokens/sec**
- Per-GPU concurrent batch (paged attention, 8K ctx): **~8–32 reqs**
- Throughput per H100 $\approx 16 \text{ reqs} \times 50 \text{ tok/s} = \text{~800 tok/sec/GPU}$
- Output capacity needed: $500B / 86,400 \approx \text{5.8M tok/sec}$
- Required GPUs (output-bound): $5.8M / 800 \approx \text{~7,250} + \text{headroom} \rightarrow \text{~10K GPUs}$
- H100 capex ~\$30K each → **~\$300M capex** for the flagship cluster

KV-Cache Memory

- Per-token KV (70B, fp16, 80 layers, GQA): **~2 MB / token / request**
- Per-request at 8K context: $8,192 \times 2 \text{ MB} \approx \text{16 GB}$
- H100 has 80 GB HBM; weights ~140 GB sharded across 2 GPUs (TP=2)
- Free HBM per GPU after weights $\approx 80 - 70 = \text{~10 GB for KV}$
- Paged attention packs many partial sequences into shared 16-token blocks

KEY

Key Numbers

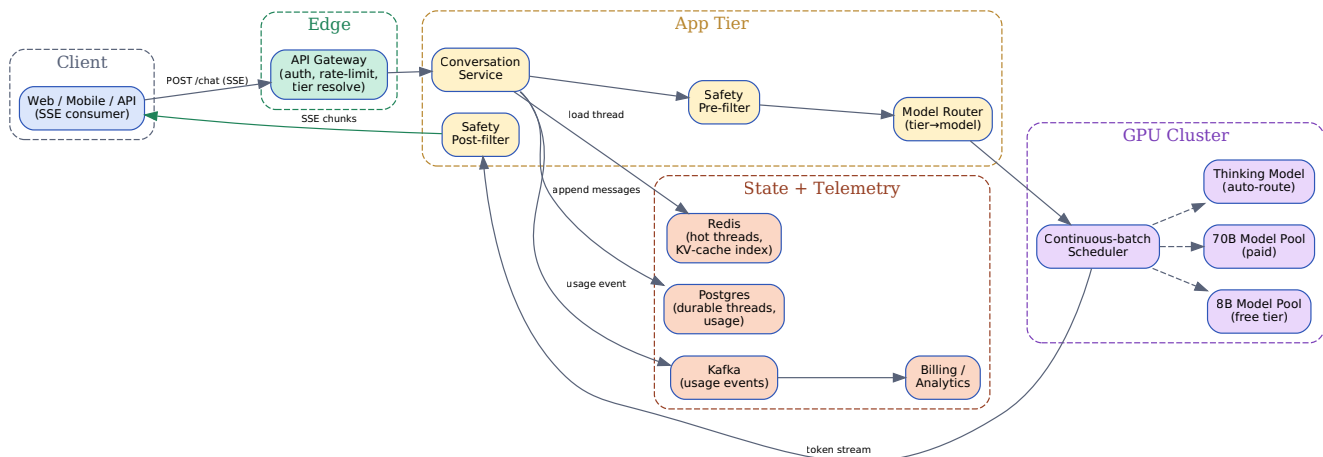
200M MAU · 1B prompts/day · 12K QPS avg / 50K peak · 500 tok response · 200 tok context · 50 tok/sec/req · 2 MB KV/tok/req · 16 GB KV @ 8K · 10K H100 cluster ≈ \$300M capex

04

High-Level Architecture

From client tap to GPU and back

User traffic enters the **API Gateway** (authentication, rate-limiting, tier resolution), is handled by the **Conversation Service** which loads thread state, runs **safety pre-filtering** and selects a model via the **Model Router**. The request then joins a continuous batch on a **Model Server**; tokens stream back through an **SSE** connection while a **safety post-filter** watches the output stream. Conversation state lives in Redis (hot) and Postgres (durable); usage events flow to Kafka for billing.



Request flow: gateway → conversation service → safety pre-filter → router → model server (continuous batching on GPU) → safety post-filter → SSE stream. Side path: Redis + Postgres for thread state; Kafka for usage and billing.

Architecture Highlights

- **API Gateway:** JWT auth, token-bucket rate-limit, resolves user tier
- **Conversation Service:** stateful coordinator; loads thread, owns SSE socket
- **Safety pre-filter:** blocks prompt-injection / disallowed content before GPU
- **Model Router:** picks 8B / 70B / thinking model; degrades on overload
- **Continuous-batch scheduler:** joins new requests into in-flight batch each step
- **Safety post-filter:** classifies streaming output; can interrupt + apologise
- **Redis:** hot conversation state; KV-cache locality hints
- **Postgres:** durable message log + usage; sharded by user_id
- **Kafka:** async usage + billing pipeline; never blocks the chat path

05

GPU Pooling & Model Placement

Sharing \$300M of silicon

Cluster Layout

- **Pods:** 8 H100 GPUs per node (NVLink); racks of 8 nodes (64 GPUs)
- **Tensor parallelism:** 70B fp16 weights → 140 GB; TP=2 fits on 2× H100
- **Pipeline parallelism:** across nodes for thinking model (200B+)
- **Replica:** a model deployment = TP-group; many replicas behind router
- **Pool sizing:** 8B pool ~30% of GPUs, 70B pool ~60%, thinking ~10%

Why Pools, Not Per-User Allocation

- Per-user GPU is wasteful: each H100 must run at >70% util to amortise capex
- Continuous batching mixes requests from different users on the same GPU step
- Pool model: route a request to the least-loaded replica that hosts the right model
- Affinity hint: send follow-up messages to the same replica to reuse prefix KV-cache

GPU Economics Cheat-Sheet

Resource	Number
H100 capex (street)	~\$30,000 per GPU
10K-GPU cluster capex	~\$300M (excludes networking, power, DC)
DGX H100 server (8 GPUs)	~\$300K
Power	~700W per H100; ~10 MW for 10K-GPU cluster
Useful life	3–4 years before next-gen replacement
Effective \$ / 1M tokens (70B)	~\$1–3 amortised at 70% util

INSIGHT

Why Utilisation Is Everything

A 10K-GPU cluster sitting at 30% utilisation costs the same as one at 90% utilisation. Continuous batching, prompt caching and KV-cache reuse exist to push that number up. A 10-point utilisation gain on a \$300M cluster is roughly \$30M of effective capacity.

06

Continuous Batching

vLLM / TensorRT-LLM style scheduling

Static vs Continuous

Aspect	Static batching	Continuous batching
Batch composition	Fixed at start, decodes together	Dynamic; new reqs join each decode step
Tail latency	Bad — fast reqs wait for slowest in batch	Good — finished slots free immediately
GPU utilisation	Drops as sequences finish	Stays high; padding minimal
Implementation	Simple	Requires paged KV-cache + iteration-level scheduler
Used by	Naive HF pipeline	vLLM, TensorRT-LLM, TGI, Triton

Continuous Batching Loop (pseudocode)

```
# Iteration-level scheduler that runs forever on each replica.
active = []           # in-flight requests
wait_q = priority_queue() # admitted but not yet started

while True:
    # 1. Admit new requests until KV-cache budget is full.
    while wait_q and kv_pool.has_room_for(wait_q.peek()):
        req = wait_q.pop()
        kv_pool.allocate_blocks(req, prefill_len=len(req.prompt))
        prefill(req)           # one big GEMM, fills KV for prompt
        active.append(req)

    if not active:
        sleep(small); continue

    # 2. One decode step for the whole active batch.
    logits = model.decode_step(active) # batched matmul on GPU
    for req, tok in zip(active, sample(logits)):
        req.append(tok)
        sse_send(req.client, tok)
        if tok == EOS or req.len >= req.max_tokens:
            kv_pool.free(req)
```

```
active.remove(req)
sse_close(req.client)
# 3. KV-cache may now have room → loop back to admission.
```

Engine Comparison

Engine	Strengths	Weaknesses
vLLM	Open-source; PagedAttention; great throughput; easy ops	Slightly behind on cutting-edge kernels; Python overhead
TensorRT-LLM	Fastest single-stream latency; FP8 / fused kernels; NVIDIA tuned	Closed-source kernels; per-model build step; harder to operate
Triton Inference Server	Multi-framework; mature ops; ensembles	Not LLM-aware on its own; needs vLLM/TRT-LLM backend for batching

07

KV-Cache Management

Paged attention and reuse

What the KV-Cache Is

- Each transformer layer caches **K** and **V** projections of every prior token so decode is $O(1)$ per token, not $O(n^2)$
- Per-token size (70B, fp16, 80 layers, GQA 8 heads): **~2 MB / token / request**
- At 8K context: $8,192 \times 2 \text{ MB} = \text{~16 GB}$ per request
- Without paging this is allocated as one giant slab → fragmentation and OOM

Paged Attention

- Borrow OS virtual-memory idea: split KV into fixed **16-token blocks**
- Each request owns a **page table** mapping logical positions → physical blocks
- Blocks live in a shared GPU pool; allocate on demand as the sequence grows
- Different requests can **share** blocks for an identical prefix (e.g. system prompt)
- Eviction policy: least-recently-used full sequence; never mid-decode

Logical Layout

```
# KV-cache page table for one request.
{
  "request_id": "req_98ab",
  "model": "flagship-70b",
  "block_size": 16,                # tokens per block
  "page_table": [
    {"logical": 0, "physical": 12048, "shared": true },
    {"logical": 1, "physical": 12049, "shared": true },
    {"logical": 2, "physical": 30771, "shared": false},
    ...
  ],
  "prefix_hash": "sha256(system+history)", # for sharing
  "ref_count": 1
}
# Shared blocks: ref_count > 1; copy-on-write if a divergent token is sampled.
```

Reuse Wins

- **Multi-turn reuse:** follow-up message keeps prior turns in cache → skip prefill for them

- **System prompt cache:** a 2K-token system prompt shared across millions of users — prefill it *once*
- **Beam / parallel sampling:** multiple candidates share the prompt blocks
- **Hash-keyed:** $\text{hash}(\text{model} + \text{tokens}) \rightarrow \text{physical block id}$; lookup before allocate

TIP

Why Paging Beats Slabs

Slab allocation reserves `max_context_len` up front for every request. If the request finishes at 200 tokens but reserved 8,192, you wasted ~16 GB. Paging only allocates blocks you actually fill, so a busy replica can serve 4-8× more concurrent requests on the same HBM.

08

Streaming Responses (SSE)

Token-by-token to the browser

Why Stream

- A 500-token reply at 50 tok/sec is a 10-second wait if buffered; unacceptable UX
- Streaming hides latency: user starts reading after <500ms TTFT
- Lets users **cancel** mid-generation and free GPU time

SSE vs WebSocket vs gRPC

Transport	Pros	Cons
SSE (chunked HTTP)	One-way; works through proxies; auto-reconnect; trivial in browser	HTTP/1.1 6-conn limit per host; no native binary
WebSocket	Bidirectional; binary; lower per-msg overhead	Harder through corporate proxies; manual reconnect
gRPC server-streaming	Strong typing; HTTP/2 multiplex; great server-to-server	Browser support requires gRPC-Web shim

SSE Wire Format

```
HTTP/1.1 200 OK
Content-Type: text/event-stream
Cache-Control: no-cache
Connection: keep-alive

event: token
data: {"t":"Hello","i":0}

event: token
data: {"t":" world","i":1}

event: usage
data: {"prompt_tok":42,"completion_tok":18}

event: done
data: [DONE]
```

Server-Side Plumbing

- Conversation Service holds the SSE socket; subscribes to a Redis channel keyed by request_id
- Model Server publishes each generated token chunk to that channel
- Cancel: client closes SSE → Conversation Service publishes cancel → scheduler removes request, frees KV blocks
- **Mid-stream migration is hard** — if the GPU pod dies, surface a clear error and let the client retry; do not try to resume

09

Conversation Memory

Threads in Redis + Postgres

Storage Tiers

- **Redis (hot):** active threads in last 24h; full message log; ~1ms reads
- **Postgres (durable):** all threads; sharded by user_id; source of truth
- **Object storage (cold):** threads > 30 days old → S3/GCS for compliance

Thread Schema

```
-- Postgres, sharded by user_id
CREATE TABLE threads (
  thread_id      UUID PRIMARY KEY,
  user_id        BIGINT NOT NULL,
  title          TEXT,
  model          TEXT,          -- last model used
  created_at     TIMESTAMPTZ DEFAULT NOW(),
  updated_at     TIMESTAMPTZ,
  msg_count      INT DEFAULT 0,
  total_tokens   BIGINT DEFAULT 0
);

CREATE TABLE messages (
  msg_id         BIGSERIAL PRIMARY KEY,
  thread_id      UUID REFERENCES threads,
  role           TEXT,          -- 'system' | 'user' | 'assistant'
  content        TEXT,
  prompt_tok     INT,
  completion_tok INT,
  model          TEXT,
  created_at     TIMESTAMPTZ DEFAULT NOW(),
  INDEX idx_thread_time (thread_id, created_at)
);
```

Read / Write Path

1. Conversation Service receives POST /v1/chat
2. GET thread from Redis; if miss, load from Postgres and warm Redis
3. Append user message to thread (write-through to both stores)
4. Compose prompt: system prompt + last N turns (token-budgeted)
5. Stream assistant response; buffer the full response

6. On done event: append assistant message; UPDATE threads SET ...; emit usage event to Kafka

Context Window Budgeting

- Reserve ~25% of model context for the response (e.g. 32K total → 8K reserved)
- Walk history newest-first; include while running token total $\leq$ budget
- Drop oldest turns; never split a single message in half
- For very long threads: produce a **running summary** with the cheaper model and prepend it

10

Tiered Routing

Free, Paid, Auto-thinking

Tiers

Tier	Default model	Limits	Cost target
Free	8B fast model	40 msgs / 3 hours	< \$0.005 / query
Plus	70B flagship	80 msgs / 3 hours	~\$0.05 / query
Pro	70B + thinking	Effectively unlimited; soft cap	~\$0.20 / query
Enterprise	Dedicated capacity	SLA + custom RPS	Negotiated

Routing Logic

- Gateway resolves `tier = user.plan`; passes through to Router
- Router has a static map: `tier → eligible model pools`
- **Auto-route** (Pro): a small classifier inspects the prompt; complex/STEM/multi-step → thinking model, simple chat → flagship
- **Overload degradation**: if 70B pool queue depth > threshold, downgrade non-Plus traffic to 8B with a note
- Sticky routing on follow-up: send to a replica that already has the prior turns' KV-cache

Thinking Model

- Generates a hidden chain-of-thought, then a final answer
- Latency budget: 10–60 seconds; GPU-cost ~5–10× of flagship
- Hide the chain-of-thought in API responses; show a UI *“thinking...”* indicator
- Use only when classifier confidence × user tier justifies it

11

Rate Limiting

Token-bucket per user, org and tier

Why Token-Bucket

- Smooth bursts (a power user firing 10 messages in 30 sec) without permanent over-allocation
- Stateless to clients; tiny state in Redis (one counter + timestamp)
- Two buckets per request: **requests/min** and **tokens/min**

Bucket Config

Tier	Req/min	Input tok/min	Output tok/min
Free	20	20K	10K
Plus	60	120K	60K
Pro	200	500K	250K
Enterprise	Negotiated	Negotiated	Negotiated

Implementation Sketch

```
# Atomic refill+decrement using a Redis Lua script.
# KEYS[1] = bucket key  ARGV = capacity, refill_rate, now, cost
LUA = ''
local data    = redis.call('HMGET', KEYS[1], 'tokens', 'ts')
local tokens  = tonumber(data[1]) or tonumber(ARGV[1])
local last_ts = tonumber(data[2]) or tonumber(ARGV[3])
local now     = tonumber(ARGV[3])
tokens = math.min(tonumber(ARGV[1]),
                  tokens + (now - last_ts) * tonumber(ARGV[2]))
if tokens < tonumber(ARGV[4]) then
    return {0, tokens}          -- 429
end
tokens = tokens - tonumber(ARGV[4])
redis.call('HMSET', KEYS[1], 'tokens', tokens, 'ts', now)
redis.call('EXPIRE', KEYS[1], 600)
return {1, tokens}             -- OK
''
```

```
# Two checks per request: rl:user:<id>:req and rl:user:<id>:tok  
# org bucket also checked when present
```

Where It Lives

- Front-line check at API Gateway → fast reject, no GPU touched
- Second check inside Conversation Service against *actual* token usage post-decode
- Output-token bucket is debited as the SSE stream progresses; can stop mid-stream and 429

12

Safety Pipeline

Pre-filter → model → post-filter

Three Layers

- **Input filter:** prompt-injection, jailbreak patterns, disallowed content (CSAM, weapons, etc.)
- **Model alignment:** RLHF-trained refusal behaviour for in-policy edges
- **Output filter:** classifier on streaming tokens; also a final policy check on the full response

Why a Separate Classifier

- Faster: a 1B-param classifier runs in <20ms on a tiny GPU
- Independently versioned: safety teams ship updates without touching the flagship
- Defence in depth: catches model failures (the model said yes; classifier says no)

Streaming Post-Filter

1. Tokens flow from Model Server to Safety Post-filter via Redis Pub/Sub
2. Post-filter buffers ~32 tokens at a time and runs the classifier
3. If a chunk scores above threshold: stop the stream, send event : blocked to client, cancel the underlying decode
4. Otherwise forward the chunk to SSE — adds <30ms p50 to TTFT

Prompt-Injection Defences

- Strip HTML / role markers (e.g. </im_start>) from user content
- Ignore instructions embedded inside tool outputs (mark them as data, not instructions)
- Run the input filter against both the new turn *and* any retrieved/pasted context
- Audit log: store {user_id, hash(prompt), classifier scores, decision} for review

WARNING

Strictness vs Helpfulness

Setting the classifier threshold too low blocks benign questions and users churn. Too high and policy violations leak. Calibrate per category (e.g. tighter on medical advice, looser on programming) and track false-positive rate as a first-class SLO.

13

Cost Optimisations

Squeezing more tokens per dollar

KV-Cache Reuse (Multi-Turn)

- Reuse the prior turns' KV blocks if they are still in HBM on the same replica
- Saves the prefill GEMM for everything except the new user message — typical multi-turn save: **30–60% of prefill cost**
- Requires sticky routing: `thread_id` → `replica` via consistent hash

Prompt Caching

- System prompts and few-shot exemplars are identical across millions of requests
- Hash the prefix; if hash already maps to allocated KV blocks, point the new request at them
- **10× reduction** in prefill cost for boilerplate-heavy assistants

Speculative Decoding

- Use a small **draft model** (e.g. 1B) to propose k tokens at a time
- Big model verifies all k in one forward pass; accepts the longest matching prefix
- Speed-up: 2–3× on natural text where draft is usually right

Speculative Decoding Sketch

```
# Speculative decoding outer loop.
while not done:
    # 1. Draft model proposes k=4 cheap tokens.
    draft = small_model.generate_k(state, k=4)

    # 2. Big model evaluates them in ONE forward pass.
    big_logits = big_model.forward(state + draft)

    # 3. Accept the longest prefix where big agrees with draft.
    accepted = []
    for i, tok in enumerate(draft):
        if argmax(big_logits[i]) == tok:
            accepted.append(tok)
        else:
            accepted.append(argmax(big_logits[i])) # correction
            break
    state += accepted
```

```
sse_send(client, accepted)
# Net effect: 1 big-model call yielded ~2.5 tokens on average.
```

MoE Routing

- Mixture-of-Experts: 200B params total, only ~20B active per token
- Router gates each token to top-2 experts; effective FLOPs $\approx$ a 20B dense model
- Memory-bound, not compute-bound: needs lots of HBM but few FLOPs
- Pairs well with continuous batching; expert imbalance is the main risk

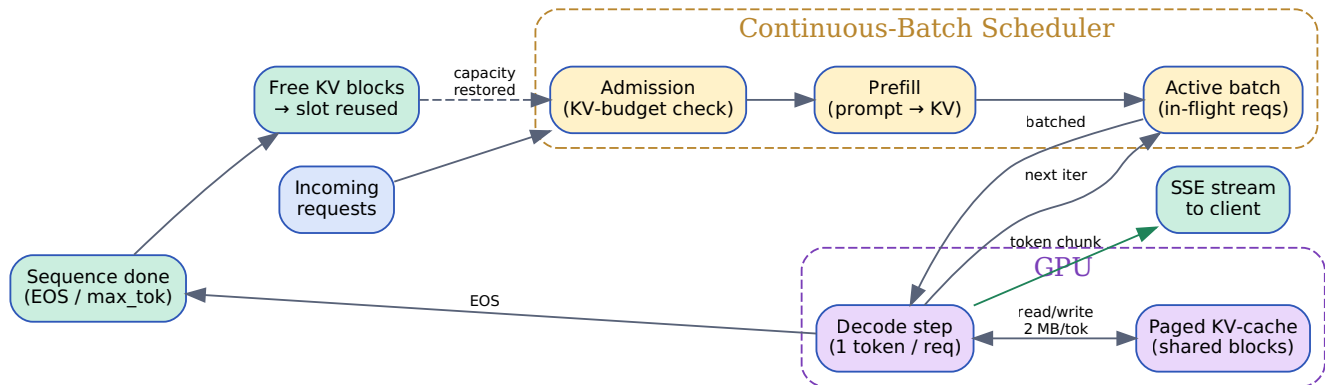
Other Levers

- **Quantisation:** fp16 $\rightarrow$ fp8 / int4 on weights; $\sim 2\times$ throughput, small quality hit
- **FlashAttention-2/3:** kernel-level memory + compute wins (free)
- **Heterogeneous fleet:** route free-tier to older A100s; flagship to H100/B200
- **Off-peak batch jobs:** evals, fine-tunes run at night when chat QPS dips

14

Inference Batching Diagram

How a request lives and dies on a GPU



Continuous-batching scheduler: incoming requests join the in-flight batch each decode step. When a sequence emits EOS its KV blocks are freed and the slot is reused. Tokens stream out as SSE.

What This Buys You

- Decode step is a single batched matmul over *all* active requests — GPU stays busy
- Sequences finish independently; free slot is filled within one iteration
- Admission control on KV-budget prevents OOM-induced full-replica crashes
- Prefill and decode share the same engine; new arrivals don't stall ongoing streams

15

Failure Modes & Trade-offs

Things that break and decisions to defend

Failure Modes

Failure	Impact	Detection	Mitigation
KV-cache OOM on long context	Active reqs killed; replica thrash	HBM > 95% gauge	Admission control on KV budget; spillover to swap is a no-go
GPU pod crash mid-stream	All in-flight streams die; SSE closes	Health check; missing heartbeats	Surface clear error to user; client retries new turn — do not try to migrate
Prompt-injection attack	Model leaks system prompt or PII	Output classifier; red-team probes	Pre-filter; treat retrieved/pasted context as data, not instructions
Thinking model latency spike	p99 blows through 60s SLA	Latency histogram alert	Fall back to flagship + apology; cap thinking budget
Hot replica (sticky routing)	One replica saturates while others idle	Per-replica QPS skew	Bound stickiness; fall back to least-loaded after threshold
Postgres shard down	Cannot persist new messages	Conn errors	Buffer in Redis; replay; serve in read-only degraded mode
Safety classifier false positive spike	Benign answers blocked; user churn	Block rate + complaint rate	Roll back classifier version; tune thresholds per category

Trade-offs

Decision	Choice	Trade-off
Latency vs throughput	Continuous batching at moderate batch size	Larger batches → higher throughput but higher TTFT; tune per pool
KV-cache eviction	Never evict an active sequence; reject new admits instead	Better tail latency for in-flight; new requests queue under load
Safety strictness vs helpfulness	Per-category thresholds + RLHF	Stricter loses users; looser leaks policy violations
	Free → 8B; Pro → 70B + thinking	

Decision	Choice	Trade-off
Paid model cost vs free experience		Free quality must be good enough to convert; paid must be worth \$20/mo
Streaming protocol	SSE	Trivial in browser; tiny per-token overhead vs WebSocket binary
State storage	Redis hot + Postgres durable	Two systems to operate; double-write race rare and reconcilable
Mid-stream resume	Not supported	Simpler design; one bad pod = one bad turn, not a cluster-wide bug

WARNING

Graceful Degradation Hierarchy

Under overload: (1) shorten max_tokens, (2) downgrade Auto/thinking → flagship, (3) downgrade flagship → 8B for free tier with a banner, (4) shed traffic with 429. Never deliver a hung connection — fail fast with a typed error so the client can retry.

16

Interview Playbook

How to present this in 45 minutes

LLM serving questions reward candidates who can do GPU economics in their head, name the right systems primitives (paged attention, continuous batching, SSE), and remember that this is fundamentally a constrained-resource scheduling problem on a \$300M cluster.

45-Minute Outline

1. **Intro (2 min):** 200M MAU, ~1B prompts/day, multi-turn, streaming, safety
2. **Capacity (6 min):** 12K QPS, 50 tok/s/req, 2 MB KV/tok, ~10K H100s, ~\$300M capex
3. **High-level arch (4 min):** Gateway → Conversation Svc → Safety → Router → GPU pool → SSE
4. **Continuous batching (8 min):** static vs continuous; iteration-level scheduling pseudocode
5. **KV-cache (6 min):** paged attention, prefix sharing, prompt caching
6. **Streaming + memory (5 min):** SSE, Redis hot + Postgres durable
7. **Routing + rate limit (4 min):** tier map, token-bucket, auto-route to thinking
8. **Safety (4 min):** pre/post filters, separate classifier, prompt injection
9. **Cost levers (4 min):** KV reuse, prompt cache, speculative decoding, MoE
10. **Failures (2 min):** KV OOM, GPU crash mid-stream, sticky-replica skew

Common Follow-ups & Answers

- **Q: How big is the KV-cache?** A: ~2 MB per token per request for a 70B model; 16 GB at 8K context. Paged attention packs partial sequences into 16-token blocks.
- **Q: Why continuous over static batching?** A: Static makes fast requests wait for the slowest; GPU util drops as sequences finish. Continuous fills slots every step → 2–4× throughput at the same latency.
- **Q: How do you stream?** A: SSE over chunked HTTP. Conversation Service holds the socket; Model Server publishes tokens via Redis Pub/Sub; Safety post-filter sits between them.
- **Q: What happens if a GPU pod dies mid-response?** A: We surface a typed error and let the client retry. Mid-stream migration is intractable because KV-cache is GPU-resident and context-specific.
- **Q: How do you protect against prompt injection?** A: Pre-filter classifier on input + retrieved context, treat tool outputs as data, hide chain-of-thought, post-filter on output.
- **Q: How do you make the free tier cheap?** A: Smaller model (8B), aggressive prompt caching, lower rate limits, off-peak capacity sharing, quantisation to int4 weights.
- **Q: Speculative decoding — when is it a loss?** A: When the draft and big model disagree often (code with tricky tokens, low-temperature precise outputs); the verification pass is wasted.

KEY

Memorize These Numbers

200M MAU · 1B prompts/day · 12K QPS avg / 50K peak · 200 tok context / 500 tok response · 50 tok/sec/req on 70B · 8–32 concurrent reqs/GPU · 2 MB KV / token / request · 16 GB KV @ 8K ctx · H100 ~\$30K · 10K-GPU cluster ~\$300M capex.